

EMBEDDED MICROCONTROLLERS

NAME: KELLY ATIENO DIANGA

**SOLAR POWERED SECURITY
STREETLIGHTS AND ALARM
LIGHTS**

Contents

PROPOSED CASE	3
BACKGROUND STUDY	3
ADVANTAGES OFFERED BY THE SYSTEM.....	3
DISADVANTAGES OF THE SYSTEM	4
CASE DESCRIPTION	4
Automatically turn on the house security lights when night time comes and off during daytime....	4
Turn on motion light to for extra brightness when mtion is detected.	4
Turn on alarm system once motion has been detected	4
Turn off the alarm system and motion light.	4
I/O TABLE.....	5
ILLUSTRATION OF PROPOSED CASE	6
FLOWCHART	7
SOURCE CODE EXPLANATION	7
Adding header Files	7
Defining Port Peripherals.....	7
Define the ADC Range.....	8
Define the minimum and Maximum Temperature and Lux that will correspond to the ADC values	8
Delay Function.....	8
Get Lux Function.....	9
Get Temperature Function	9
Enable clock to ADC and Ports	9
Configure Port Pins.....	9
Set Direction of Pins	10
Set the temperature and Lux from which the sensors will react to	10
Set conditions to which the sensors will react.....	10
Display when lights are on or off as well the sensor readings	11
Use for loop to run the program infinitely.....	12
FULL SOURCE CODE.....	12
RESULTS	15
Turning off Security lights and Alarm Lights	15
Turning on Security lights and Alarm Lights.....	15
Turning on Security lights only	16
Turning on alarm lights only	17

Controlling the sensitivity of the PIR sensor for the alarm lights.....	18
Controlling the sensitivity of the PIR sensor for the alarm lights.....	19
FUTURE CODE EXPANSION.....	19
Using a timer for the alarm system.	19
Control button to control the brightness and dimness of the lights.....	19
WEBLINK TO DEMONSTRATION VIDEO	20
REFERENCES.....	20

PROPOSED CASE

Using a microcontroller for home security light system.

BACKGROUND STUDY

The security lights in this report refers to the solar streetlights we can install in our home compounds for security purposes. These lights are built to automatically switch on (and use their stored electricity) come dusk, and to switch off (and begin generating electricity again) come dawn. They are able to do this by using sensors which detect the loss of light, and automatically trigger the ON and OFF switch of the lights. For this case, we will use the microcontroller to control the lights by triggering the on and off switch of the lights by reacting to the sensors. For this case, we will use the Lux sensor which is able to detect the time of day by measuring the amount of sunshine present in the unit lux.

To further enhance safety in our homes, motion detector sensors can be used to detect random motion in certain portions of our compounds/ inside our homes. This then triggers an alarm system which turns on an additional set of lights for extra brightness. The PIR sensor can also be used for the home lighting to turn on or off lights in different rooms whenever motion is detected. We will also use the microcontroller to control the alarm lights by reacting to the readings from the PIR motion sensor.

ADVANTAGES OFFERED BY THE SYSTEM

Some of the advantages offered by the system are;

- No electricity bills while using the streetlights as they store absorb the solar energy during the day and store it in the batteries and using the stored energy to light the place at night.
- Environmentally friendly. Solar streetlighting uses renewable solar energy which does not drain the planet of its finite energy resources.
- No power outages. By using the solar streetlights you can be sure that your home compound will be kept safe and well-lit even in the event of a blackout or loss of grid power!
- Cost effective. PIR sensors are cost effective and simple to install. Solar streetlights also use solar energy to operate saving on a lot of money that might otherwise be used for the electricity bill.

- Energy saving. By having a system that automatically turns on and off, we are able to save energy that could be lost whenever we forget to turn off the lights when we leave our rooms.
- Very convenient. Imagine walking into a dark room with a bag full of groceries and fumbling with the switch in the darkness! Having a system that turns on lights when you enter a room and off when you leave the room proves to be very convenient for such cases.
- Sense of Security. By having a system that alerts you by whenever the presence of an intruder is detected creates a sense of much needed security.

DISADVANTAGES OF THE SYSTEM

Just as there are pros, the system also has some cons. This is listed below.

- Expensive purchase. Whilst solar energy products tend to save you money in the long run, they can often require an expensive initial investment.
- Reliance on light. Whilst some monocrystalline solar cells do continue generating electricity in low light, most (if not all) solar street lights will still require several hours every day of fairly direct sunlight in order to achieve their maximum potential.
- False alert. As the motion detector security system are sensitive to motion, they will end up giving false alerts whenever an animal for example is detected.

CASE DESCRIPTION

In this assignment, we will use the microcontroller to;

Automatically turn on the house security lights when night time comes and off during daytime

Sunlight is monitored with a LUX sensor that provides a voltage in the range of 0-3.3V covering the LUX from T_0 in the linear fashion. The security lights are switched on at set LUX L with a 4 Hysteresis. Switch on at $(L-2)$ and off at $(L+2)$. Security light operation is emulated by an LED. Set L is initially .

Turn on motion light to for extra brightness when motion is detected.

Motion is monitored using a PIR motion sensor that also provides a voltage on the range of 0 – 3.3V, covering the temperature from T_0 . Sensor is emulated by a potentiometer. Sensor switched on at temperature T with 4 degrees hysteresis i.e switch on at $(T-2)$ and off at $(T+2)$. Sensor operation is emulated by LED. Set T is initially .

Turn on alarm system once motion has been detected

When motion is detected, just as with the motion lights, the alarm system is also activated .

Turn off the alarm system and motion light.

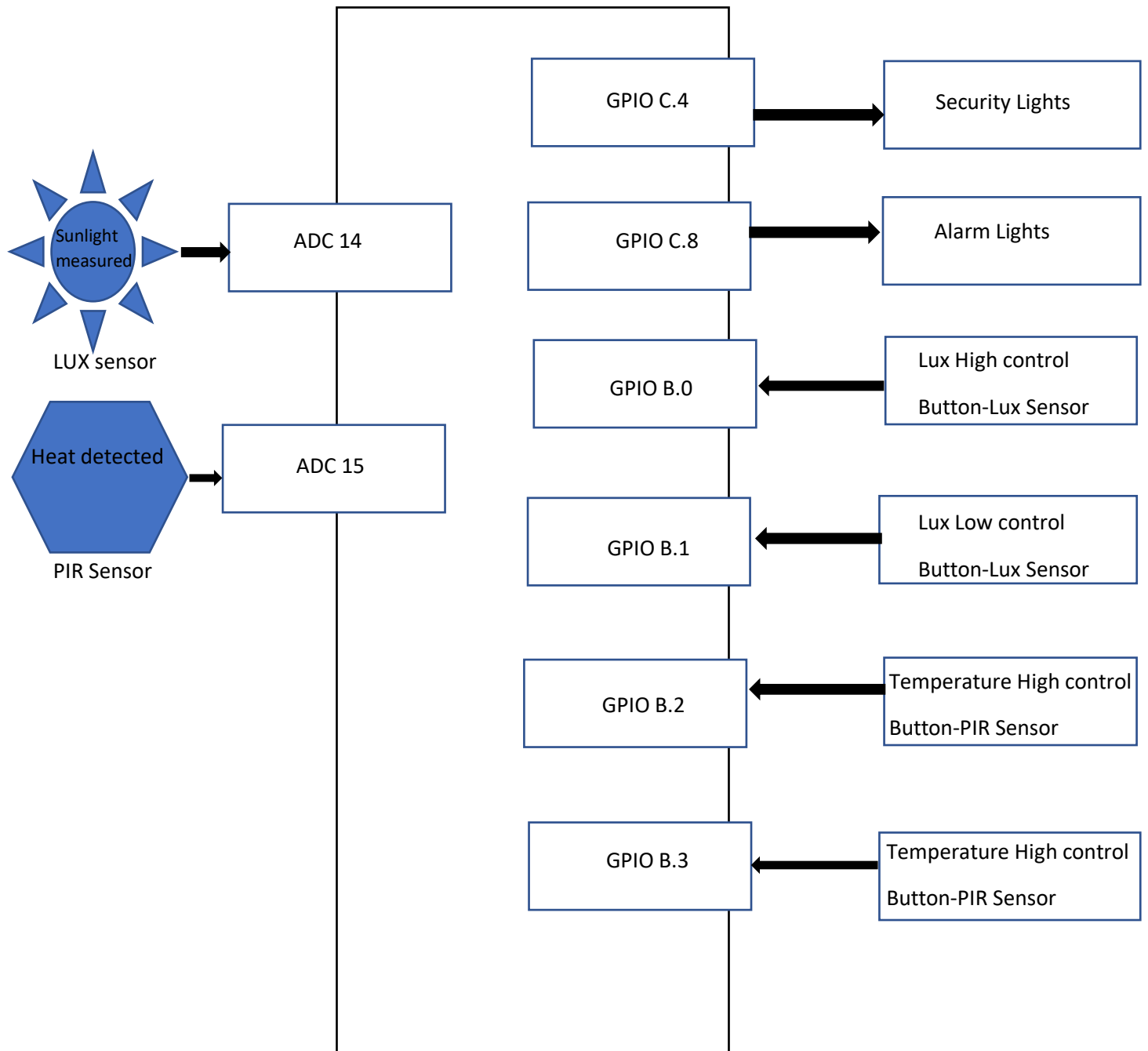
The alarm switch is used to turn off both the alarm system and motion light.

I/O TABLE

The table below shows the input and output port pins and channels that will be used in this case.

NAME	BIT	TYPE	FUNCTION
Security Light	1	Output	Shows that the security lights of the home are turned on at night and off during the day
Motion Detector Light	2	Output	Switched high when the movement is detected
Alarm Light	3	Output	Switched high when alarm is triggered . Remains high until alarm is disabled.
LUX sensor control buttons	0,1	Input	Controls the sensitivity of the lux sensor by increasing or decreasing the set lux
PIR motion sensor control buttons	2,3	Input	Controls the sensitivity of the PIR sensor by increasing or decreasing the set temperature.
Alarm Switch	4	Input	Used to disable alarm and motion light.
Sunlight	14		ADC Channel will be used as a potentiometer that reflects the amount of sunlight present.
Room Temperature	15		ADC Channel will be used as a potentiometer that reflects the temperature/heat present.

ILLUSTRATION OF PROPOSED CASE

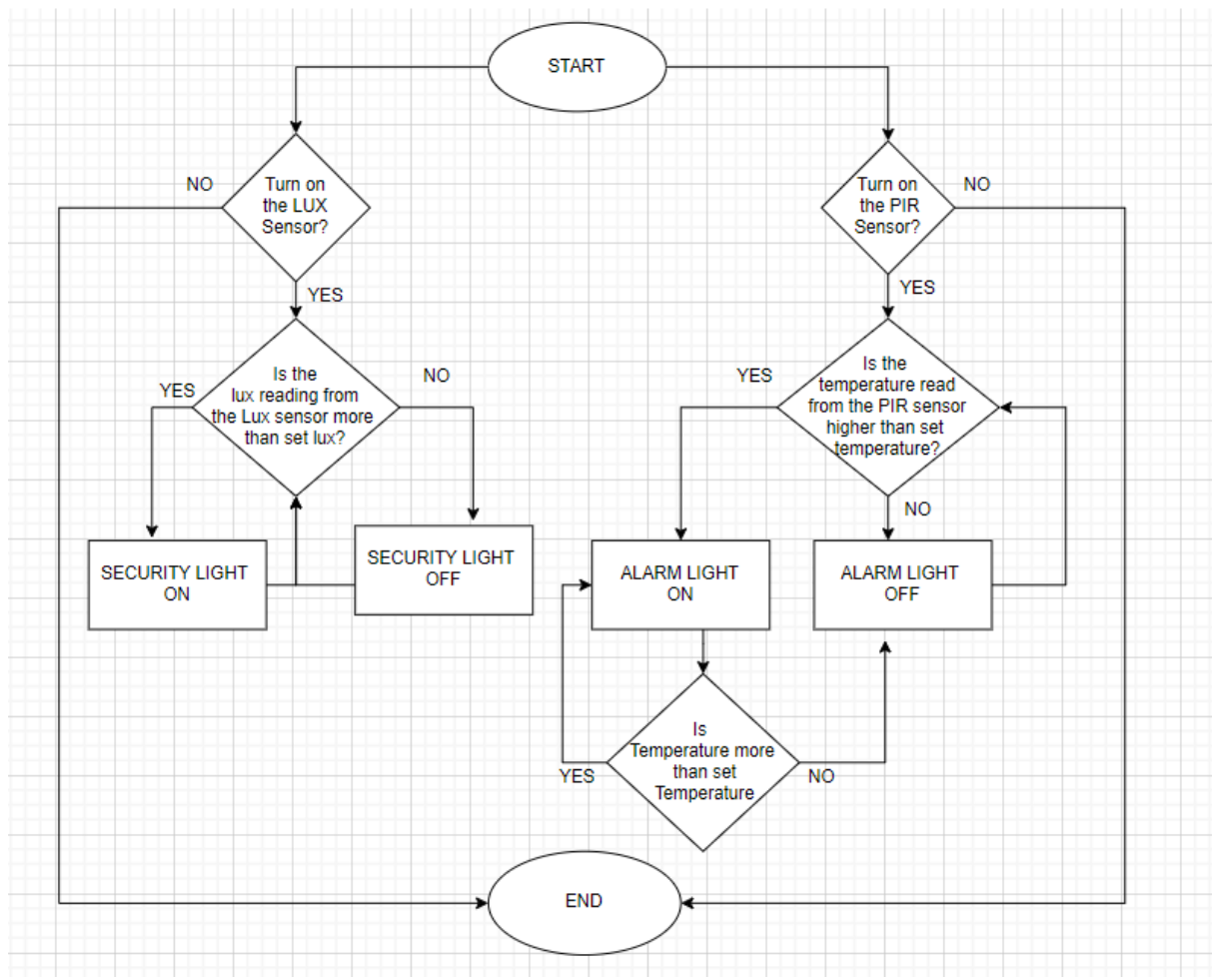


The arrow direction shows whether its an input or output. If pointing out, it is an output. If arrow is pointing inwards, its an input.

For this case,we shall assume the Lux Sensor can measure lux value from 0 to 65000 lux.

We shall aslo assume that the PIR sensor can take reading from 10 to 50 degrees.

FLOWCHART



SOURCE CODE EXPLANATION

Adding header Files

To access the registers and declare suitable macros that will map to the registers in C program, I added the “derivative.h” header file.

To be able to write and print out the messages in the termite, I included the header files “clock.h” and “stdio.h”.

```
/* INCLUDE HEADER FILES*/
#include "derivative.h"
#include <stdio.h>
#include "clock.h"
```

Defining Port Peripherals

To run this program, used 4 port B pins (0,1,2,3) for as input. These are the control buttons used to control the sensitivity of the sensors by increasing or decreasing the set temperature and Lux.

I used 2 port C pins (4 and 8) for the security and alarm Lights respectively. These are connected to the LEDs to show when the lights are on or off.

I used ADC channel 14 as a potentiometer to show how the changes in voltage affect the security lights. Voltage reading here reflect the converted digital reading taken by the LUX sensor

I used ADC channel 15 as a potentiometer to show how the different voltages affect the alarm clock. Voltage reading here reflect the converted digital reading taken by the PIR motion sensor

```
/* DEFINE PERIPHERALS */
//Security and Alarm light
#define securityLight 1<<4 //PTC4
#define alarmLight 1<<8 //PTC8

//Control knob for LUX Sensor
#define sensitivityHighMask 1<<0 //PTB0
#define sensitivityLowMask 1<<1 //PTB1

//CONTROL knob for PIR motion sensor
#define tempUpMask 1<<2 //PTD2
#define tempDownMask 1<<3 //PTD3

//ADC for LUX AND Temperature reading conversion
#define luxADC 14 //ADC Channel 14
#define tempADC 15 //ADC Channel 15

/* Hysteresis to apply to LUX and temperature changes */
#define HYSTERESIS 2
```

Define the ADC Range

I used a 10-bit resolution for our analogue to digital conversion for more accurate results by reducing the quantization error.

```
/** Range of ADC values ( ADC resolution - 10-bit) */
#define ADC_MAX ((1<<10)-1)//1023
```

Define the minimum and Maximum Temperature and Lux that will correspond to the ADC values

I did some scaling for the conversion by defining the minimum values of the lux and temperature that corresponded to the minimum voltage reading in the ADC channel as well as the maximum values of the lux and temperature that corresponded to the maximum voltage reading in the ADC channel.

```
/* Minimum LUX and temperature corresponding to minimum ADC value (0) */
#define LUX_MIN 0
#define TEMP_MIN 10

/* Maximum LUX and temperature corresponding to maximum ADC value (ADC_MAX) */
#define LUX_MAX 65000
#define TEMP_MAX 50
```

Delay Function

I used this function to create some delay so as to be able to see the changes taking place as the analogue to digital conversion occurs for different temperature and lux sensor readings.


```

/* A simple counting loop to waste some time so as to see the changes being made.*/
void delay() {
    int delayCount;
    for(delayCount = 0; delayCount < 1000000; delayCount++){
        asm("nop"); // No - operation but don't delete
    }
}

```

Get Lux Function

This function was used by the microcontroller to convert the readings obtained from the LUX sensor into digital voltage readings in ADC channel 14.

```

/* Function that obtains the input LUX in the range 0 to 65000 LUX and returns digital value of LUX*/
unsigned int getLUX() {
    //Trigger conversion on ADC Channel 14
    ADC0_SC1A = ADC_SC1_ADCH(luxADC);

    //Wait for conversion to complete
    while((ADC0_SC1A & ADC_SC1_COCO_MASK)==0);
    // Scale ADC value to LUX
    return LUX_MIN + ((unsigned int)ADC0_RA * (LUX_MAX-LUX_MIN))/ADC_MAX;
}

```

Get Temperature Function

This function was used by the microcontroller to convert the readings obtained from the PIR motion sensor into digital voltage readings in ADC channel 15.

```

/* Function that obtains the input temperature in the range 10 to 50 degrees and returns converted temperature*/
unsigned int getTemp() {
    //Trigger conversion on ADC Channel 15
    ADC0_SC1A = ADC_SC1_ADCH(tempADC);
    //Wait for conversion to complete
    while((ADC0_SC1A & ADC_SC1_COCO_MASK)==0);
    // Scale ADC value to temperature
    return TEMP_MIN + ((unsigned int)ADC0_RA * (TEMP_MAX-TEMP_MIN))/ADC_MAX;
}

```

MAIN FUNCTION CODES

Enable clock to ADC and Ports

The Port and ADC clocks act as a master switch,i.e no code written will be performed by the Port or ADC registers when their respective clocks are off. Therefore,before accessing any of the ADC registers as well as Port pins, I turned on their respective clocks.

```

int main(void) {
    clock_initialise();
    // Enable clock to ADC
    SIM_SCGC6 |= SIM_SCGC6_ADC0_MASK;
    // Enable clock to PORT C and PORT B
    SIM_SCGC5 |= SIM_SCGC5_PORTC_MASK | SIM_SCGC5_PORTB_MASK;
}

```

Configure Port Pins

I manipulated the functions of the ADC channel and Port pins to suit the use for this program.

For the ADC channels,I set them to low power operation(restrict maximum clock speed) with a conversion resolution of 10 bits.

For the port pins used for the LED lights, I set them as GPIO pins with a high drive strength.

For the port pins used as control buttons, I set them as GPIO pins with the internal resistors of an upward a pull direction.

```
// Set to Low Power and 10-bit conversion
ADC0_CFG1 = ADC_CFG1_ADLPC_MASK | ADC_CFG1_MODE(2);
// Configure Port pins
PORTC_PCR4 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK; //Security Lights
PORTC_PCR8 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK; //Alarm Lights
PORTB_PCR0 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK; //Higher Sensitivity
PORTB_PCR1 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK; //Lower Sensitivity
PORTB_PCR2 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK; //Higher Temperature
PORTB_PCR3 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK; //Lower temperature
```

Set Direction of Pins

I set the Pin directions for Port C as output as they will be used for the security and alarm Ligts.

I then set the pin direction of Port B as input as they will be used for the control button to adjust the sensitivity of the sensors.

```
// Set Security and Alarm light Pins Direction (OUTPUT)
GPIOC_PDDR |= securityLight;
GPIOC_PDDR |= alarmLight;

// Set control knobs Pin Direction (INPUT)
GPIOB_PDDR &= ~(sensitivityHighMask | sensitivityLowMask);
GPIOB_PDDR &= ~(tempUpMask | tempDownMask);
```

Set the temperature and Lux from which the sensors will react to

I also set the temperature and Lux values for which the sensors will react to. If the sensor readings obtained are less or more than the set reading, the microcontroller will be prompted to turn the security and alarm lights on or off.

```
//Set the temperature and LUX at which the sensors will react to deviations from.
unsigned int setLUX = 15000;
unsigned int setTemperature = 20;
```

Set conditions to which the sensors will react

I used if statements to set the conditions for which the sensors will react to.

For the LUX sensor

As the amount of LUX indicates the amount of sunlight present, the high lux read shows presence of sunlight and vice versa. Therefore, when the amount of lux read is lower than the set LUX, the security lights will be turned on. If the amount of lux read by the sensor is more than the set lux, then the security lights will be turned off.

The lower the set LUX, the higher the sensitivity. Therefore, to increase the sensitivity of the sensor, reduce the set lux, to decrease the sensitivity, increase the set lux.

```

/*SECURITY LIGHTS OPERATION
-----*/
unsigned lux = getLUX();// converted input LUX read from the sensor is assigned to the variable LUX

/* Check LUX bound*/
if (lux >= (setLUX+HYSTERESIS)) {
    GPIOC_PCOR = securityLight; //Switch off security light if input LUX is more than set LUX
}
if (lux <= (setLUX-HYSTERESIS)) {
    GPIOC_PSOR = securityLight; //Switch on security lights if input LUX is less than set LUX
}
printf("Input LUX= %3d, Set LUX = %3d, Security Lights = %s\n", lux,
setLUX, (GPIOC_PDIR & securityLight)?"On":"Off\n");//Show input LUX, set LUX and every time security light is turned on or off

/*LUX Sensor Sensitivity Buttons
 * Increasing sensitivity means reducing the amount of LUX needed for the sensor to react to
 * Decreasing Sensitivity means increasing the amount of light the sensor needs to react to.
 */
if ((GPIOB_PDIR & sensitivityHighMask) == 0 && setLUX >= 5000) {
    while((GPIOB_PDIR & sensitivityHighMask) == 0);
    setLUX -= 500;//Increase Sensitivity by 500 LUX every time high sensitivity button is pressed
}
if ((GPIOB_PDIR & sensitivityLowMask) == 0 && setLUX <= 20000) {
    while((GPIOB_PDIR & sensitivityLowMask) == 0 );
    setLUX += 500;//Decrease Sensitivity by 500 every time low sensitivity button is pressed
}
delay();

```

For the PIR Sensor

As the amount of heat detected indicates the presence or absence of a person., the sudden increase in temperature read shows presence of motion. Therefore, when the temperature read is higher than the set LUX, the alarm lights will be turned on. If the temperature read by the sensor is less than the set lux, then the alarm lights will be turned off.

As the sensor is sensitive to the environmental temperature, those living in areas experiencing winter and summer seasons will need to adjust the set temperature for more accuracy. So during warm days and nights, it is advisable to increase the temperature set for more accuracy and vice versa.

```

/*ALARM LIGHTS OPERATION
-----*/
unsigned temperature = getTemp();// converted input Temperature read from the sensor is assigned to the variable temperature

/* Check temperature bounds */
if (temperature >= (setTemperature+HYSTERESIS)) {
    GPIOC_PSOR = alarmLight; //if temperature detected is more than set Temperature, turn on alarm Lights
}
if (temperature <= (setTemperature-HYSTERESIS)) {
    GPIOC_PCOR = alarmLight; //if temperature detected is less than set Temperature, turn off alarm Lights
}
printf("Input Temperature = %3d, Set Temperature = %3d, Alarm Lights = %s\n", temperature,
setTemperature, (GPIOC_PDIR & alarmLight)?"On":"Off");//Show input Temperature, set Temperature and every time alarm light is turned on or off

/*PIR Sensor Control Buttons*/
if ((GPIOB_PDIR & tempUpMask) == 0) {
    while((GPIOB_PDIR & tempUpMask) == 0);
    setTemperature++;
}
if ((GPIOB_PDIR & tempDownMask) == 0) {
    while((GPIOB_PDIR & tempDownMask) == 0);
    setTemperature--;
}

delay();//allow for changes taking place to be seen.

```

Display when lights are on or off as well the sensor readings

To check that the program is running as intended, I wrote code to report the reading taken by the sensors as well as the changes in the lighting to the console.

```

printf("Input LUX= %3d, Set LUX = %3d, Security Lights = %s\n", lux,
setLUX, (GPIOC_PDIR & securityLight)?"On":"Off\n");//Show input LUX, set LUX and every time security light is turned on or off

printf("Input Temperature = %3d, Set Temperature = %3d, Alarm Lights = %s\n", temperature,
setTemperature, (GPIOC_PDIR & alarmLight)?"On":"Off");//Show input Temperature, set Temperature and every time alarm light is turned on or off

```

Use for loop to run the program infinitely

I used a for loop to run the program infinitely and allow the sensors to collect readings multiple times during the day and night.

```
//Use for loop to allow the program to run infinitely
```

```
for(;;) {
```

```
    /*SECURITY LIGHTS OPERATION*/
```

FULL SOURCE CODE

```
/* INCLUDE HEADER FILES*/
```

```
#include "derivative.h"
```

```
#include <stdio.h>
```

```
#include "clock.h"
```

```
/* DEFINE PERIPHERALS */
```

```
//Security and Alarm light
```

```
#define securityLight 1<<4 //PTC4
```

```
#define alarmLight 1<<8 //PTC8
```

```
//Control knob for LUX Sensor
```

```
#define sensitivityHighMask 1<<0 //PTB0
```

```
#define sensitivityLowMask 1<<1 //PTB1
```

```
//CONTROL knob for PIR motion sensor
```

```
#define tempUpMask 1<<2 //PTD2
```

```
#define tempDownMask 1<<3 //PTD3
```

```
//ADC for LUX AND Temperature reading conversion
```

```
#define luxADC 14 //ADC Channel 14
```

```
#define tempADC 15 //ADC Channel 15
```

```
/* Hysteresis to apply to LUX and temperature changes */
```

```
#define HYSTERESIS 2
```

```
/** Range of ADC values ( ADC resolution - 10-bit) */
```

```
#define ADC_MAX ((1<<10)-1)//1023
```

```
/* Minimum LUX and temperature corresponding to minimum ADC value (0) */
```

```
#define LUX_MIN 0
```

```
#define TEMP_MIN 10
```

```
/* Maximum LUX and temperature corresponding to maximum ADC value (ADC_MAX) */
```

```
#define LUX_MAX 65000
```

```
#define TEMP_MAX 50
```

```
/* A simple counting loop to waste some time so as to see the changes being made.*/
```

```
void delay() {
```

```
    int delayCount;
```

```
    for(delayCount = 0; delayCount < 1000000; delayCount++){
```

```
        asm("nop"); // No - operation but don't delete
```

```
    }
```

```
}
```

```
/* Function that obtains the input LUX in the range 0 to 65000 LUX and returns digital value of LUX*/
```

```

unsigned int getLUX() {
    //Trigger conversion on ADC Channel 14
    ADC0_SC1A = ADC_SC1_ADCH(luxADC);

    //Wait for conversion to complete
    while((ADC0_SC1A & ADC_SC1_COCO_MASK)==0);
    // Scale ADC value to LUX
    return LUX_MIN + ((unsigned int)ADC0_RA * (LUX_MAX-LUX_MIN))/ADC_MAX;
}

/* Function that obtains the input temperature in the range 10 to 50
degrees and returns converted temperature*/
unsigned int getTemp() {
    //Trigger conversion on ADC Channel 15
    ADC0_SC1A = ADC_SC1_ADCH(tempADC);
    //Wait for conversion to complete
    while((ADC0_SC1A & ADC_SC1_COCO_MASK)==0);
    // Scale ADC value to temperature
    return TEMP_MIN + ((unsigned int)ADC0_RA * (TEMP_MAX-TEMP_MIN))/ADC_MAX;
}

int main(void) {
    clock_initialise();
    // Enable clock to ADC
    SIM_SCGC6 |= SIM_SCGC6_ADC0_MASK;
    // Enable clock to PORT C and PORT B
    SIM_SCGC5 |= SIM_SCGC5_PORTC_MASK | SIM_SCGC5_PORTB_MASK;
    // Set to Low Power and 10-bit conversion
    ADC0_CFG1 = ADC_CFG1_ADLPFC_MASK | ADC_CFG1_MODE(2);
    // Configure Port pins
    PORTC_PCR4 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK; //Security Lights
    PORTC_PCR8 = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK; //Alarm Lights
    PORTB_PCR0 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK;
//Higher Sensitivity
    PORTB_PCR1 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK;
//Lower Sensitivity
    PORTB_PCR2 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK;
//Higher Temperature
    PORTB_PCR3 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK;
//Lower temperature

    // Set Security and Alarm light Pins Direction (OUTPUT)
    GPIOC_PDDR |= securityLight;
    GPIOC_PDDR |= alarmLight;

    // Set control knobs Pin Direction (INPUT)
    GPIOB_PDDR &= ~(sensitivityHighMask | sensitivityLowMask);
    GPIOB_PDDR &= ~(tempUpMask | tempDownMask);

    //Set the temperature and LUX at which the sensors will react to
    deviations from.
    unsigned int setLUX = 15000;
    unsigned int setTemperature = 20;

    //Use for loop to allow the program to run infinitely
    for(;;) {

        /*SECURITY LIGHTS OPERATION
        -----*/
        unsigned lux = getLUX(); // converted input LUX read from the sensor
        is assigned to the variable LUX

```

```

    /* Check LUX bound*/
    if (lux >= (setLUX+HYSTERESIS)) {
        GPIOC_PCOR = securityLight; //Switch off security light if input
LUX is more than set LUX
    }
    if (lux <= (setLUX-HYSTERESIS)) {
        GPIOC_PSOR = securityLight; //Switch on security lights if input
LUX is less than set LUX
    }
    printf("Input LUX= %3d, Set LUX = %3d, Security Lights = %s\n", lux,
setLUX, (GPIOC_PDIR & securityLight)?"On":"Off\n");//Show input LUX,
set LUX and every time security light is turned on or off

    /*LUX Sensor Sensitivity Buttons
    * Increasing sensitivity means reducing the amount of LUX needed for
the sensor to react to
    * Decreasing Sensitivity means increasing the amount of light the
sensor needs to react to.
    * */
    if ((GPIOB_PDIR & sensitivityHighMask) == 0 && setLUX >= 5000) {
        while((GPIOB_PDIR & sensitivityHighMask) == 0);
        setLUX -= 500;//Increase Sensitivity by 500 LUX every time high
sensitivity button is pressed
    }
    if ((GPIOB_PDIR & sensitivityLowMask) == 0 && setLUX <= 20000) {
        while((GPIOB_PDIR & sensitivityLowMask) == 0 );
        setLUX += 500;//Decrease Sensitivity by 500 every time low
sensitivity button is pressed
    }
    delay();
    /*ALARM LIGHTS OPERATION
    -----*/
    unsigned temperature = getTemp();// converted input Temperature read
from the sensor is assigned to the variable temperature

    /* Check temperature bounds */
    if (temperature >= (setTemperature+HYSTERESIS)) {
        GPIOC_PSOR = alarmLight;//if temperature detected is more than
set Temperature,turn on alarm Lights
    }
    if (temperature <= (setTemperature-HYSTERESIS)) {
        GPIOC_PCOR = alarmLight;//if temperature detected is less than
set Temperature,turn off alarm Lights
    }
    printf("Input Temperature = %3d, Set Temperature = %3d, Alarm Lights
= %s\n", temperature,
setTemperature, (GPIOC_PDIR & alarmLight)?"On":"Off");//Show input
Temperature, set Temperature and every time alarm light is turned on or off

    /*PIR Sensor Control Buttons*/
    if ((GPIOB_PDIR & tempUpMask) == 0) {
        while((GPIOB_PDIR & tempUpMask) == 0);
        setTemperature++;
    }
    if ((GPIOB_PDIR & tempDownMask) == 0) {
        while((GPIOB_PDIR & tempDownMask) == 0);
        setTemperature--;
    }

    delay();//allow for changes taking place to be seen.

```

```

    }//end for loop
    return 0;
} //end main function

```

RESULTS

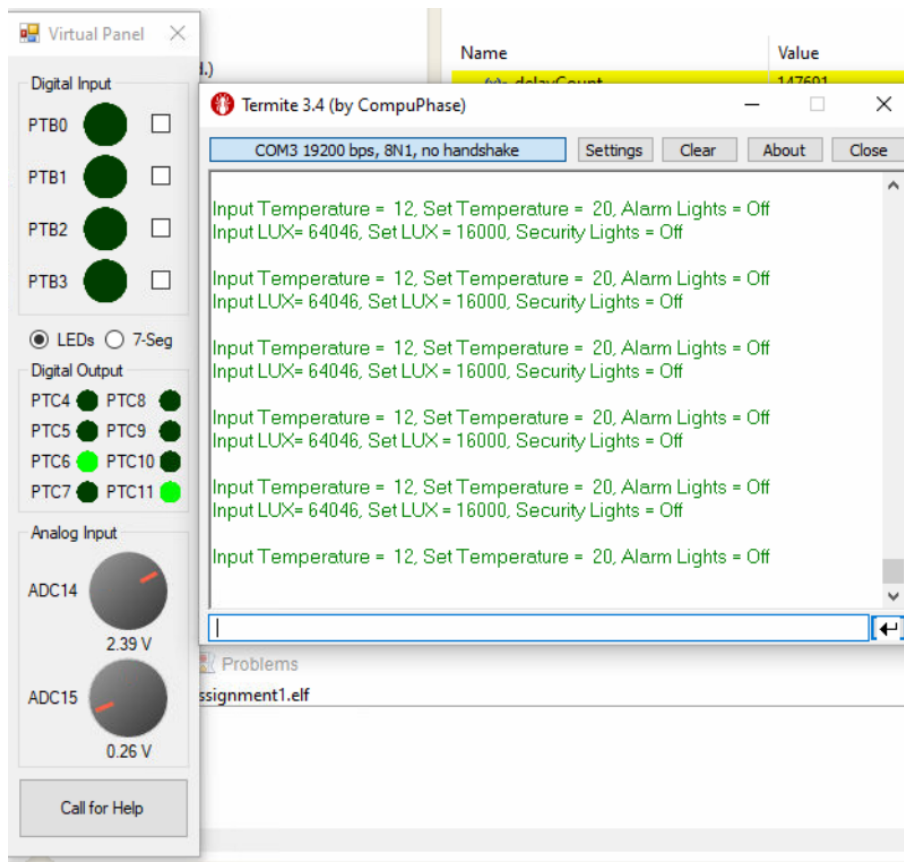
Turning off Security lights and Alarm Lights

For the lights, we are using;

Port C pin 4-Security Lights

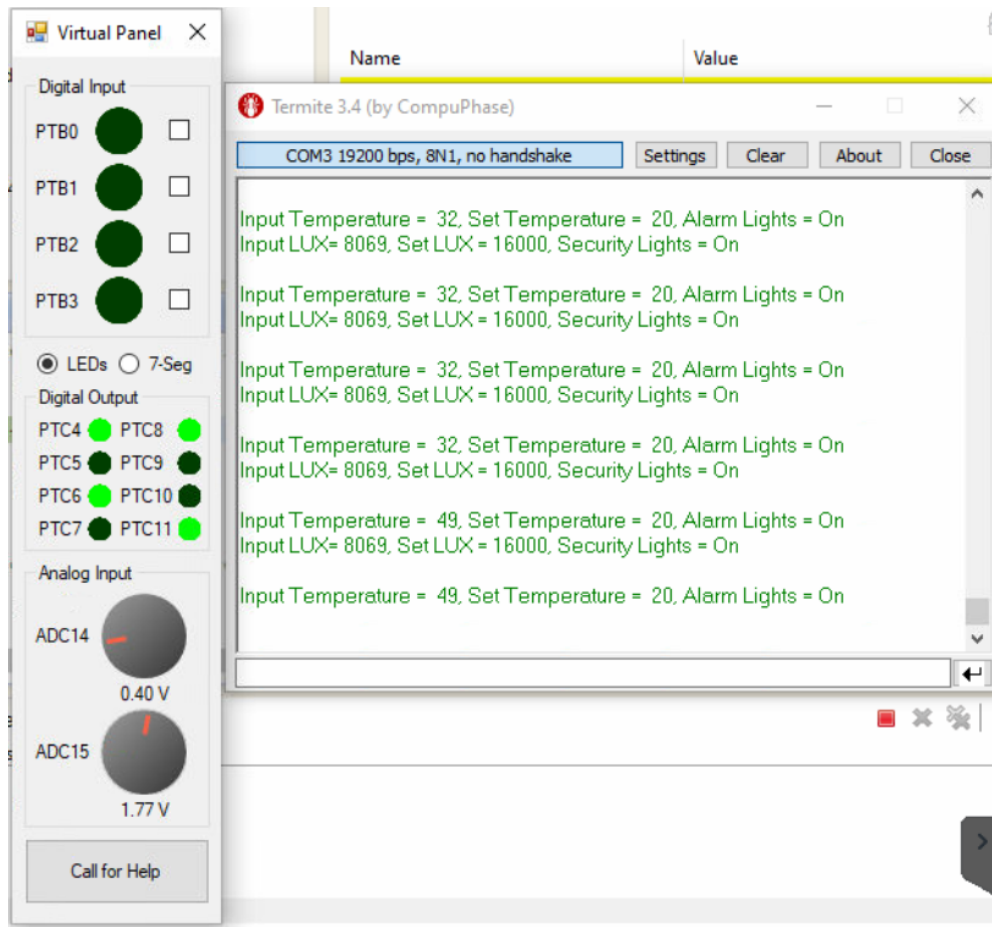
Port C pin 8- Alarm Lights

As shown from the screenshot below, both port pins are off and the message shown tells us that both lights are off. Just as expected, if the input Temperature is less than the set temperature, the alarm lights will be off and when the input lux read is more than the set lux, the security lights are off.



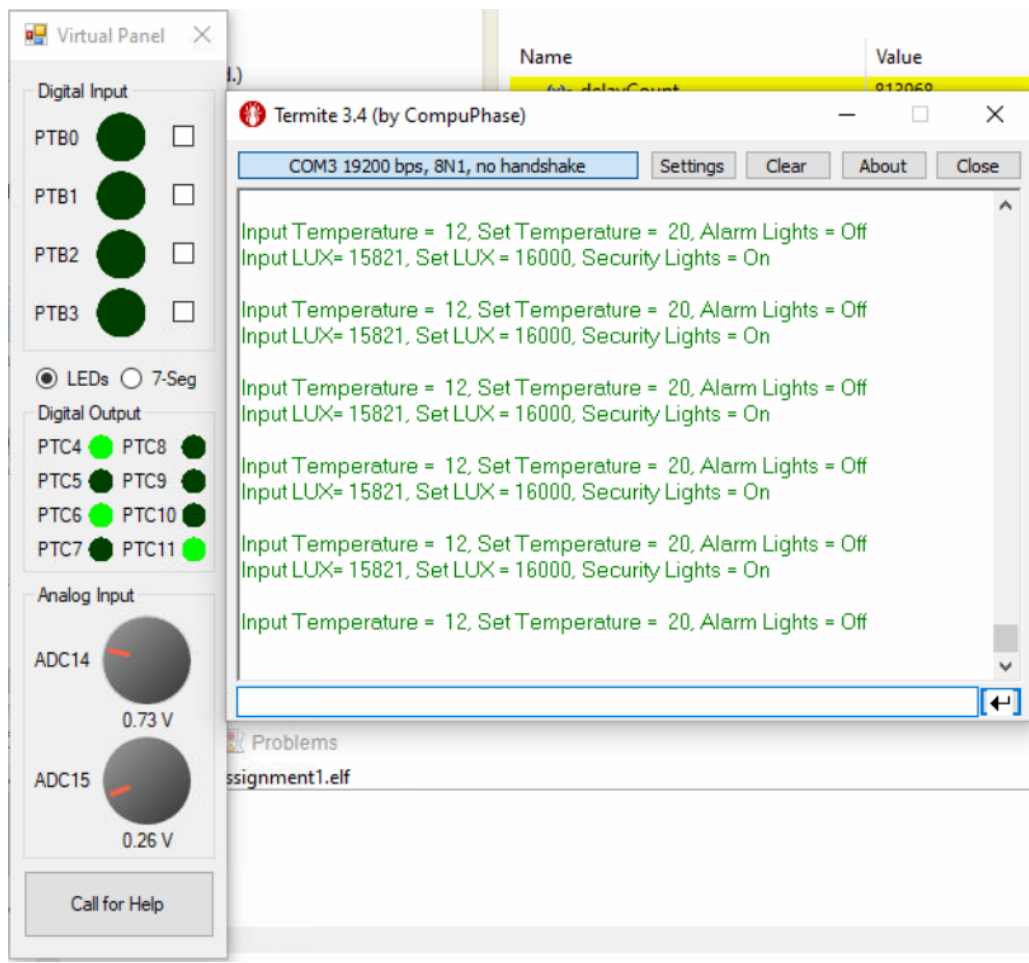
Turning on Security lights and Alarm Lights

To turn on the security lights, the input LUX needs to be less than the set LUX. To turn on the alarm lights, the input temperature needs to be more than the set temperature. This is shown in the screenshot below, whereby both port C pin 4 and 8 are turned on.



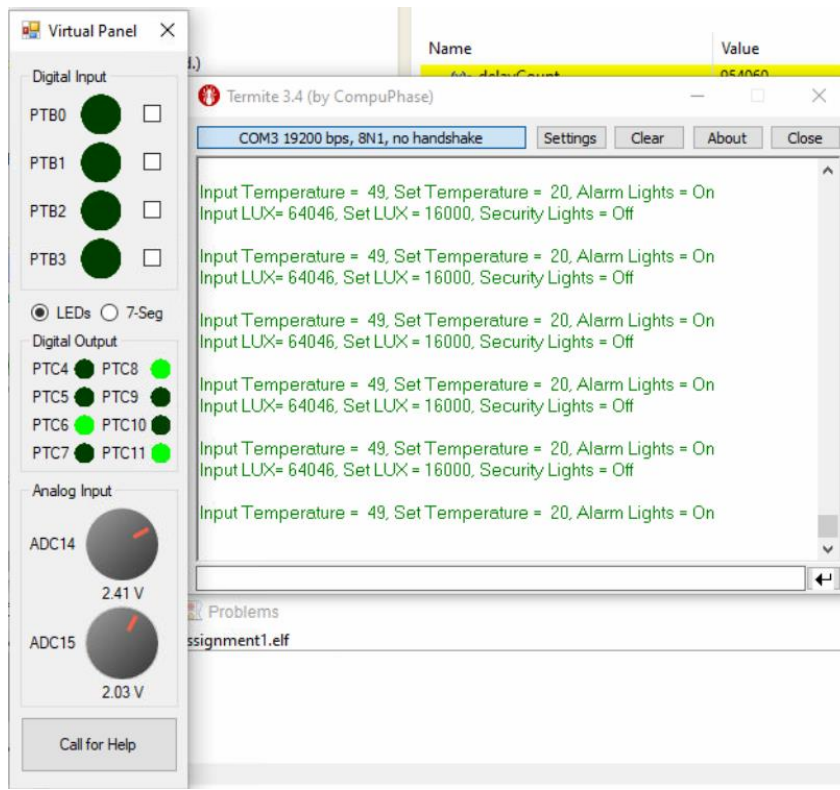
Turning on Security lights only

As expected, the security lights can be switched on separately and only turn on when input lux is less than the set lux. This is shown in the screenshot below whereby port C pin 4 is turned on below.



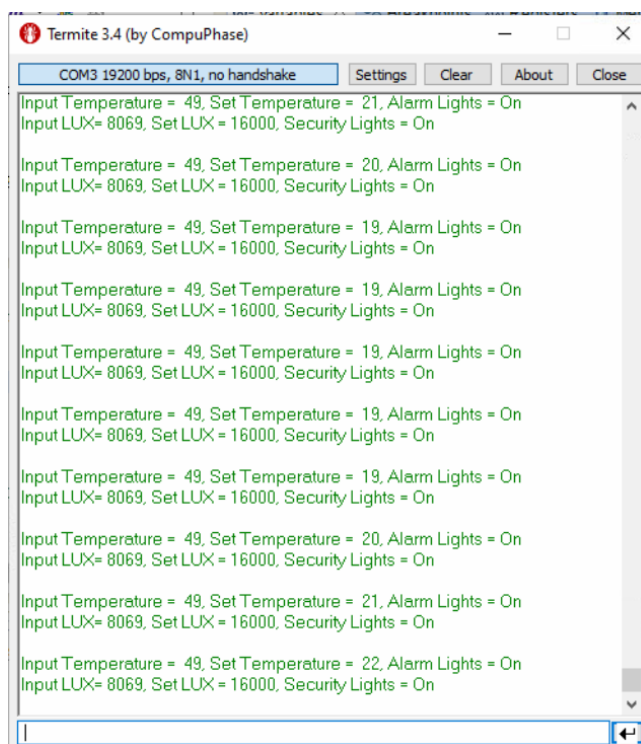
Turning on alarm lights only

Just like security lights, alarm lights can be switched on separately and only turn on when input temperature is less than the set temperature. This is shown in the screenshot below whereby port C pin 8 is turned on below.



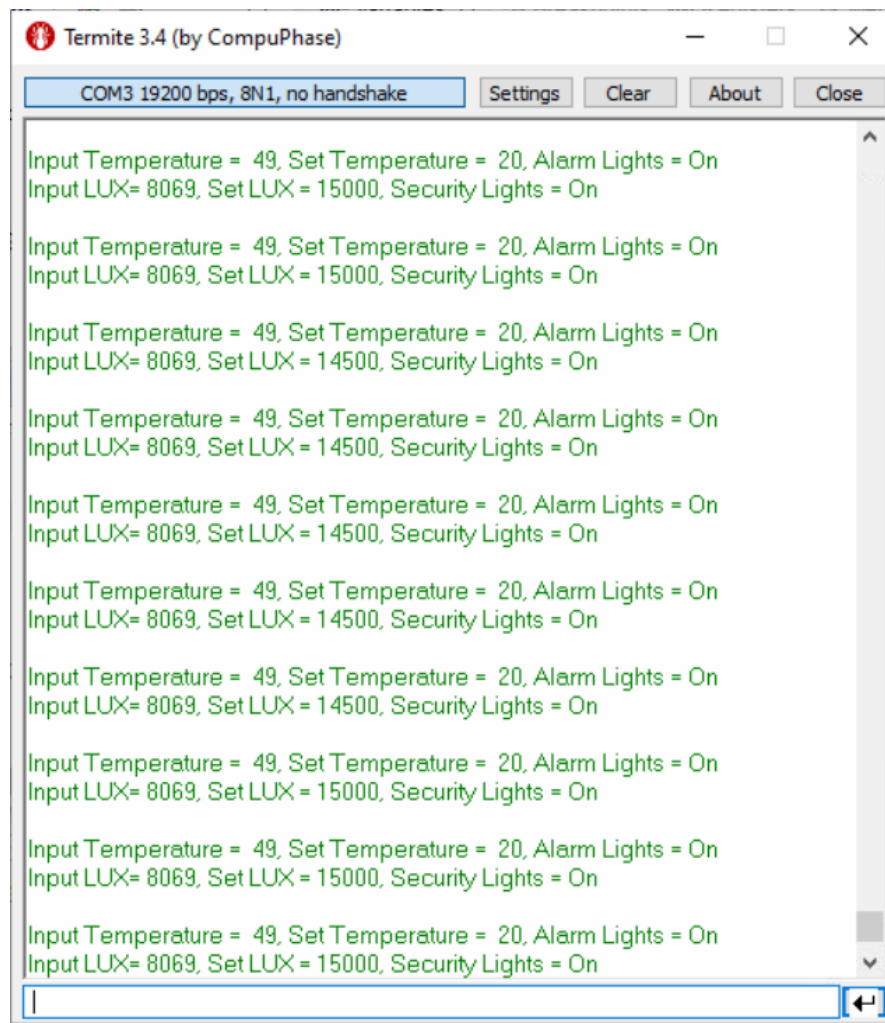
Controlling the sensitivity of the PIR sensor for the alarm lights

Sensitivity of the PIR sensor is done by adjusting the set temperature. The screenshot below shows the adjustments made to the set temperature using Port B pins 2 and 3.



Controlling the sensitivity of the PIR sensor for the alarm lights

Sensitivity of the Lux sensor is done by adjusting the set Lux. The screenshot below shows the adjustments made to the set lux using Port B pins 0 and 1.



FUTURE CODE EXPANSION

The program above only functions on the basic level. Some of my future improvements to the system includes;

Using a timer for the alarm system.

I plan on adding a timer whereby the alarm lights remain turned on for 30 seconds before automatically turning off.

Control button to control the brightness and dimness of the lights.

I also plan on adding two more buttons to adjust the brightness of the security and alarm lights.

WEBLINK TO DEMONSTRATION VIDEO

<https://youtu.be/seNpxiuU9CQ>

REFERENCES

[motion-sensor-for-security-light-using-pir-sensor.pdf \(ijmter.com\)](#)

[How Do Solar Street Lights Work? \(szsolarlight.com\)](#)

[Length of sunlight using a lux sensor? - Using Arduino / Sensors - Arduino Forum](#)